NEMA 17 / MKS SERVO42D — Bring-Up SOP

Arctos Robotic Arm — joints A, B and C

September 2026

Follow these steps in order to take a NEMA 17 joint from powered-off to trusted for motion. Every step has a pass condition; if one fails, stop there rather than continuing.

Background, protocol details and troubleshooting live in `Arctos_CAN_Interface.pdf`. Bare terminal commands live in `BASIC_CAN_COMMANDS.pdf`.

Which joint you have

Joint	CAN ID	Gear ratio	Role
A	`0x04`	48.0:1	Elbow
B	`0x05`	67.82:1	Wrist rotation
C	`0x06`	67.82:1	Wrist joint

Caution: Confirm the ID by scan, and reprogram the panel if it disagrees

The table above is the intended assignment. A controller's panel can hold a different value: on 2026-09-02 Joint B was found answering on `0x06`, which is Joint C's ID, because that driver's panel had been saved wrong.

If the scan in Step 3 disagrees with this table, **go back to Step 1 and reprogram the panel** rather than carrying on against the wrong ID. Two joints sharing an ID will collide, and a frame sent to the wrong ID drives the wrong joint.

Wherever a command below shows `0x06` and `67.82`, substitute your own joint's CAN ID and gear ratio.

Step 0 — Before applying power

1. Confirm the controller is the **CAN variant** of the MKS SERVO42D. Read the part number on the board. The RS485 variant has no CAN transceiver and will never respond, whatever you do in software.
2. Everything powered off, measure CAN_H to CAN_L. Expect about 60 Ω.
3. Confirm only CAN_H, CAN_L and GND run between the CANable and the controller. Do not connect the CANable's power lines.

4. Confirm the joint is mechanically clear, and that you know its travel limits if it is mounted on the arm.

Pass: About 60 Ω across the bus, and a confirmed CAN-variant board.

If it fails: 120 Ω means only one terminator is present. An open circuit means broken wiring. Fix it before continuing.

Step 1 — Configure the controller panel

Power the controller and set these on its own display, then **save**:

CAN ID	the ID for this joint
CAN Rate	`500`
CAN Response, shown as CANRSP	`ENABLE`
CAN CRC	`SUM-8`
Working current	1.2 A or below, the NEMA 17's rating
Working mode	`SR_vFOC` — see the caution below

Power-cycle the controller after saving. The CAN ID in particular has to be committed, not merely displayed.

Caution: Set the working current deliberately

A controller in this project was found storing a 3000 mA default against a motor rated 1.2 A, roughly two and a half times over. Check this on every new controller.

Caution: Set the working mode to `SR_vFOC`, not a `CR_` mode

The controller ships in `CR_vFOC`. The `CR_` modes drive the motor from the physical pulse interface (`STP`/`DIR`/`En`); the `SR_` modes drive it from serial and CAN commands, which is how this arm is controlled. `arctos_interface.cpp`'s `setupMotorParameters()` sends `SET_WORKING_MODE (0x82)` with `SR_vFOC` (value 5) to every joint, so the panel should agree with the stack rather than be left on the factory default.

Two things go wrong otherwise. A mode mismatch **imitates a communication fault**: the controller accepts a frame and acknowledges its checksum without turning the shaft, so a "no motion" result looks like wiring or CAN when it is neither. And **endstops cannot work in a `CR_` mode** on this build — limit port remap puts the limits on `En` and `Dir`, which in a `CR_` mode are the pulse interface's own enable and direction inputs. The vendor manual restricts the remap to serial control mode for exactly that reason.

The mode can also be set over CAN rather than on the panel:

```
# SET_WORKING_MODE (0x82) = SR_vFOC (5), CAN ID 0x06
# checksum 0x06 + 0x82 + 0x05 = 0x8D
cansend can0 006#82058D
```

Pass: Settings survive the power cycle.

Step 2 — Bring up the CAN interface

```
lsusb
ls -l /dev/serial/by-id/
sudo pkill slcand
sudo ip link delete can0 2>/dev/null
sudo slcand -o -c -s6 /dev/serial/by-id/usb-Openlight_Labs_CANable2* can0
sudo ip link set can0 txqueuelen 1000
sudo ip link set up can0
ip -br link show can0
```

Pass: The last command prints `can0` followed by `UP`.

If it fails: Re-check the device name; it changes whenever the adapter re-enumerates. `bitrate 0` in `ip -details` output is normal on slcan and is not a fault.

Step 3 — Confirm the real CAN ID

Never assume the ID from a config file or a table.

```
cd ~/arctos_ws/src/arctos_can_control/arctos_can_control
python3 can_id_scan.py --channel can0 --ids 1-16
```

This sends only the read-only `0x31` query. It never enables coils and never commands motion, so it is safe at any time.

Pass: Exactly one responder, and its ID is the one you set in Step 1.

If it fails: No responders: re-check the panel settings, the wiring, and the board's part number. Several responders: another driver is powered on the bus with a colliding ID. Resolve that before going further.

Step 4 — Read-only health check

With the ID confirmed, check the link is trustworthy before commanding anything.

```
python3 joint_reliability_test.py --channel can0 --can-id 0x06 \
    --gear-ratio 67.82 --samples 200
```

With the joint at rest, expect:

Query	Opcode	Expected
Encoder	`0x31`	stable; zero jitter, zero drift
Velocity	`0x32`	zero
Position error	`0x39`	small, tens of counts
Enable state	`0x3A`	`01`; these drivers power up enabled
Shaft protection	`0x3E`	`00`

Pass: 100% response rate, zero checksum failures, zero encoder jitter at rest.

If it fails: Any checksum failure or dropped response is a wiring or bitrate problem. Do not proceed to motion.

Step 5 — Check the mechanism by hand

Do this before the motor ever drives the joint. It separates mechanical faults from electrical ones, which the bus on its own cannot distinguish.

1. Disable the coils. For `0x06`:

```
cansend can0 006#F300F9
```

2. Confirm the state is now `00`:

```
cansend can0 006#3A40
```

3. Turn the joint output by hand, gently, both directions, watching the encoder.

Caution: Only drop coils if the joint is not gravity-loaded

On an assembled arm this removes holding torque and the joint may fall. Support it, or skip this step if it cannot be done safely.

Pass: The encoder tracks your hand in both directions and the motion feels smooth. This proves the gearbox transmits and that nothing is skipping.

If it fails: Gritty or catching resistance means the gear mesh needs a plastic-safe PTFE or silicone grease. No encoder movement while the output turns means the motor shaft is not following: check the coupling, the grub screw and the bore. A hard stop at one repeatable angle is usually a misassembled gear rather than roughness — on Joint B a gear seated in the wrong position produced a hard limit at exactly the same angle on eight consecutive attempts.

Step 6 — First commanded motion

Paste your joint's emergency stop into a second terminal, ready but unsent, before you send anything. For `0x06`:

```
cansend can0 006#F7FD
```

Re-enable the coils, then command one small move:

```
python3 joint_unstick_move_test.py --can-id 0x06 --gear-ratio 67.82 \
    --step-degrees 0.25 --num-steps 1 --max-unstick 0 \
    --band-lo -1 --band-hi 1
```

Caution: Always set the band explicitly

Some joints have a `home_position` and an `opposite_limit` of `0.0` in `arctos_controller.yaml`, which produces an inverted and unusable safe band. Pass the band on the command line yourself, every time.

Pass: The step reports about 100% of commanded and the status frames show `FD=[1, 2]`. Both frames matter: `FD 01` on its own means the command started but never finished.

If it fails: Do not retry blindly — that only heats the motor. Go back to Step 5.

Step 7 — Stepped travel test

Extend to real travel, one direction and then the other. Keep the step size and count inside the joint's safe range.

```
python3 joint_unstick_move_test.py --can-id 0x06 --gear-ratio 67.82 \
    --step-degrees 2.0 --num-steps 9 --max-unstick 0 \
    --band-lo -25 --band-hi 25 --speed 25
```

Then the same distance back, to close the loop:

```
python3 joint_unstick_move_test.py --can-id 0x06 --gear-ratio 67.82 \
    --step-degrees -2.0 --num-steps 9 --max-unstick 0 \
    --band-lo -25 --band-hi 25 --speed 25
```

Pass: Every step 99–101% of commanded with `FD 02`, and the round trip returns to the starting position within a few encoder counts.

If it fails: A step that under-delivers followed by a step that delivers nothing means the joint is against something, or the motor has lost drive. Stop and diagnose. Do not increase torque and do not retry.

Step 8 — Sweep test, if the mesh is rough

Only needed if Step 7 showed a rough patch that a hand could push through. This sweeps back and forth across the spot for a fixed time, backing off and re-attempting whenever a step under-delivers.

```
python3 joint_sweep_unstick_test.py --can-id 0x06 --gear-ratio 67.82 \
    --sweep-lo -10 --sweep-hi 10 --step-degrees 1.0 \
    --backoff-degrees 0.5 --band-lo -25 --band-hi 25 --duration-s 150
```

Pass: Zero unstick cycles used, and every step 99–101%.

If it fails: If the joint stops at the *same* angle every time, that is a mechanical limit, not roughness — open the gearbox rather than working at it. Repeated back-and-forth through a hard particle embeds it and wears printed teeth.

Step 9 — Measure the speed ceiling

```
python3 joint_speed_limit_test.py --can-id 0x06 --gear-ratio 67.82 \
    --arc-lo -24 --arc-hi 24 --band-lo -25 --band-hi 25 \
    --speeds 100,150,175,200,250 --accel 8 --pause-s 15
```

The usable limit is the highest commanded speed where actual rpm still tracks it to within about 1%.

Caution: Keep the duty cycle down

Running a long ladder of trials back to back overheats the motor. Fifteen seconds between trials is reasonable; one second is not. Stop as soon as you have the answer, and put a hand on the motor part way through — there is no temperature opcode, so nothing in software will warn you.

Measured result for Joint B, through its 67.82:1 gearbox:

Tracks commanded within 1% up to	150 rpm
Roll-off begins	between 150 and 165 rpm
Saturation ceiling	about 165 rpm

A bare Servo42D on the bench tracks to 225 rpm and saturates near 310. A geared joint saturates well below that, so measure each joint rather than inheriting the bench figure.

Step 10 — Acceptance

The joint is ready for use when all of these hold:

Check	**Criterion**
CAN ID	confirmed by scan
Comms	100% response, zero checksum failures
Encoder at rest	zero jitter, zero drift
Hand check	smooth, encoder tracks both directions
Per-step accuracy	99–101% of commanded
Status frames	`FD 02` on every step
Round trip	returns to start within a few counts
Shaft protection	still `00`
Speed limit	measured and written down

Step 11 — Operating limits

- **Stay at or below the measured saturation speed.** Commanding more buys no extra speed and only adds current and heat.
- **Use `0xFD` for all motion.** Do not use `0xF5`.
- **Never retry into a stall.** Back off and re-attempt once, or stop.

- **Disable coils between tests**, unless the joint is gravity-loaded, in which case leave it holding.
- **Never enable shaft protection** (`0x88`). It has a history of false positives and latches the driver.

Endstops — wiring and verification

Only if this joint has hall endstops fitted. The 42D has one limit input by default and this build uses two, so it needs limit port remap.

Wire the sensor's supply to `EVCC/EGND` (the `5V` and `GND` terminals on the CAN-side block, 5 V at 20 mA), the two outputs to `En` and `Dir` on the opposite block, and `Com` to 5 V. Then enable the remap:

```
cansend can0 006#9E01A5
```

Watch the pins while passing a magnet, with the joint stationary:

```
python3 joint_endstop_test.py --can-id 0x06
```

Pass: Resting byte `0x03`; one pole drops it to `0x02`, the other to `0x01`, and it returns to `0x03` between passes.

If it fails: Nothing moves. Check `Com` is wired, the work mode is `SR_vFOC`, and that the remap command was actually sent — it has no read-back, so it cannot be confirmed by query.

Caution: On this build, enabling the remap is a routine step, not a diagnosis

Every 42D on this arm — Z, A, B and C — needs limit port remap, because the endstops are wired to `En` and `Dir` rather than to the `IN_1` terminal. **Send `9E 01` as part of bringing up a 42D** rather than waiting to discover it. Three separate joints have presented the identical signature first — sensor LED lighting, `0x34` frozen at `0x01` — and each time it cost a session to re-derive.

Note also that the setting can come back off on its own: Joint C's remap was enabled on 2026-09-03 and found disabled on 10-30 with nobody having changed it. **Check it at the start of every session.** There is no read-back, so the only test is to toggle it and watch the byte move.

Caution: `EndLimit` is not what makes an endstop readable

`EndLimit` governs only whether the driver *acts* on a limit. `0x34` reports the pin regardless, so if an input is not reporting, enabling `EndLimit` will not change it. On a 42D the missing piece is almost always the remap above — this has caught two joints now, C and Z, with the identical signature of a lit sensor LED and `0x34` frozen at `0x01`.

Once the pins report, record the joint's travel and the sensor-to-input mapping:

```
python3 joint_calibrate.py z
```

Run it twice at the same speed. Approach speed biases the trigger position from both ends at once, so a single figure is not a result.

Caution: Home with a search, not `GoHome`

`GoHome (0x91)` takes its direction from `Hm_Dir`, which cannot be read back, and the manual says the motor keeps running until it hits the limit switch. Search instead, watching `0x34` yourself, record the trigger, step clear of the detection zone, then `SET_AXIS_ZERO (0x92)`. Note that the zero does not survive a power cycle.

Caution: A parked magnet chatters

A magnet left at the edge of the sensor's range oscillates across its threshold. Use `joint_endstop_dwell.py` to confirm a deliberate hold registers as one clean event before relying on the limit; adjust sensor position before fitting the belt.

Step 12 — Shutdown

Stop any running script with Ctrl+C first.

```
cansend can0 006#F300F9
sudo ip link set can0 down
sudo pkill slcand
```

Then unplug the adapter. Skip the first line if the joint is gravity-loaded and needs to keep holding.